Tool Support for Causal Reasoning via String Diagram Surgery

Sifei Li and Fabio Zanasi

University College London, United Kingdom

Abstract. Causal and counterfactual reasoning have recently been given a category-theoretic semantics based on string diagrams and their graphical manipulation ("surgery"), yet no software supports this approach. We present a Julia prototype performing counterfactual identification on probabilistic models represented as string diagrams, with a scoped extension for causal inference computation. This lets us compare the categorical approach, conceptually and empirically, with existing R and Python implementations: an explicit diagrammatic intermediate representation achieves competitive performance while offering structural transparency and interpretability.

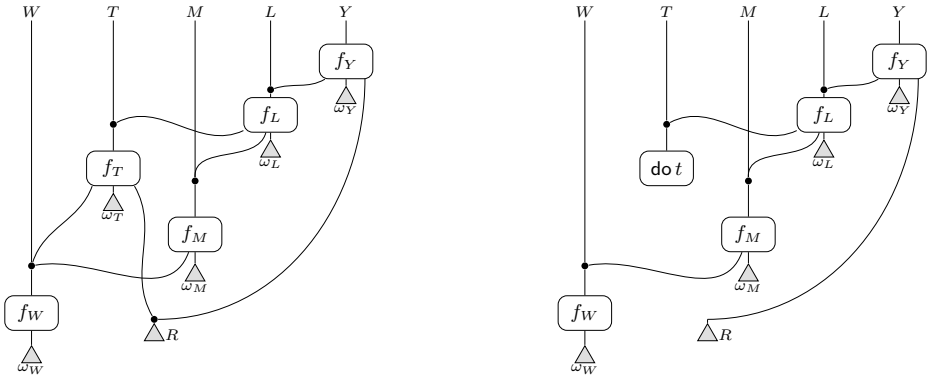
1 Introduction

Causal inference studies how interventions change the behaviour of complex systems [14], answering questions such as "what is the effect of a drug on recovery?" that statistical association alone cannot. Pearl formalises an intervention through the *do*-operator: $P(Y \mid \mathrm{do}(X{=}x))$ is the distribution of Y when X is *forced* to x rather than merely observed to equal it. Computing such effects hinges on *identification*: can the target effect be expressed in terms of the observable distribution at all? Hidden confounding can make this impossible; the ID algorithm and its counterfactual extension ID-CF [16] characterise the identifiable cases.

Counterfactuals are more expressive still: they ask how the outcome would have changed had X been set to x, given that $X = x'$ (possibly alongside further evidence) was actually observed. Such questions are central to explanation, responsibility attribution, policy evaluation, and fairness [14].

Recent work, initiated by [7] and developed in [8,13,18,4], gives causal inference a compositional semantics via category theory—notably *Markov categories* [5,9] and *string diagrams* [15]. Here a Bayesian network is not merely a graph but a morphism in a symmetric monoidal category, drawn as a string diagram whose wires are variables and boxes conditional mechanisms. Pearl's do-operator is interpreted by *string diagram surgery* [7]: the box producing the intervened variable is cut out and replaced, making intervention a local graphical rewrite, not a global symbolic substitution. The same primitive drives both queries we target: interventional effects (causal inference) and counterfactuals, where the model is duplicated into parallel worlds sharing their latent sources [8].

Since the same diagrammatic primitives recur across model construction, intervention, and simplification, this semantics is a natural basis for tool support.



(a) The commute model.

(b) After surgery do(T=t).

Fig. 1: (a) Commute model as a string diagram, read bottom to top, with confounding $T \leftrightarrow Y$ as the shared latent R; (b) its surgery do(T=t) replaces f_T by the state $\mathsf{do}\, t$, severing T from its causes.

Yet no system performs causal reasoning—interventional-effect computation and counterfactual identification—directly on this semantics: existing projects offer complementary functionality, such as categorical probabilistic libraries [2], quantum-circuit tooling [11], and general string diagram manipulators (e.g. [1,10]), but a dedicated pipeline is still missing. In this work we take the first steps to address this gap:

- We implement causal reasoning by string diagram surgery in Julia (Catlab): counterfactual identification, plus a scoped comb disintegration extension for interventional effects, with export to the string diagram rewriting tool Chyp [10]; our tool is publicly available, see [12].
- We analyse the challenges of implementing categorical semantics, including diagram representation, parallel-world construction, and simplification.
- We compare against existing R and Python tools, identifying the strengths and current limitations of the categorical approach.

2 Categorical Background

We recall just enough of the diagrammatic semantics of causal models to phrase the queries our tool answers, following the string diagram approach of [8,7]: each notion is introduced on one running example—a commuter's daily journey—with the category theory kept to the minimum needed to read the diagrams.

Bayesian Networks as String Diagrams. Figure 1a renders our running *commute* model as a string diagram (read bottom to top), its five observable variables labelling the wires: weather W, departure time T, mode of transport M, lateness L, and work performance Y. Throughout, capitals X, Y, Z denote observable variables, identified when labelling wires with their finite sets of

possible values; lower-case x, y, z denote particular values. Each variable X is produced by a *mechanism* $f_X \colon \big(\bigotimes_{V \in \mathrm{pa}(X)} V\big) \otimes \Omega_X \longrightarrow X$, drawn as a box: it reads X's direct causes (its parents $\mathrm{pa}(X)$) together with a private noise source ω_X (an exogenous state, drawn as a triangle, supplying a distribution over the noise space Ω_X and modelling everything left unmodelled), and outputs a distribution over X; the boxes $f_W, \ldots, f_Y$ are the local causal mechanisms of the five variables above. At a glance the diagram resembles a Bayesian network, but unlike a network it is an *algebraic* object: read as a morphism of a symmetric monoidal category (SMC), it can be reasoned about compositionally. In this reading, variables are objects, and $\otimes$ is the monoidal product, acting on value spaces as their Cartesian product; boxes are morphisms, assembled by sequential composition and monoidal product (respectively, vertical and horizontal juxtaposition); states such as $\omega_X : I \to \Omega_X$ are morphisms with no input, i.e. out of the monoidal unit I. Each object also carries a *copy* map (black dot) duplicating a value and a *discard* map deleting it; an SMC with these is a *copy–discard* (or Markov) category [5,9], a basic framework for probabilistic computation; a copy dot is inserted wherever a variable feeds several mechanisms. *Confounding* (a hidden common cause, written $T \leftrightarrow Y$) is a single latent state R copied into both f_T and f_Y: for the commuter, an unobserved factor (say, recurring engineering works) influences both when she sets off and how she performs.

Causal Inference via String Diagram Surgery. Suppose we ask how the commuter's work performance Y would change if her departure time were *set* to t—the interventional effect $P(Y \mid \mathrm{do}(T{=}t))$. Categorically, the intervention is modelled via an endofunctor, rendered graphically as *string diagram surgery* ([8,7,13]): the mechanism f_T is cut out and replaced by a state $\mathsf{do}(T{=}t) : I \to T$ fixing T to t (the point distribution at t), recording that T is now *forced* rather than caused (Figure 1b). Severing T from its causes while leaving the rest intact yields a diagram denoting $P(\,\cdot \mid \mathrm{do}(T{=}t))$. The effect is then recovered from the diagram's existing conditionals by a *comb disintegration*, whose details we omit ([8]); this procedure is the causal-inference component of our tool (Section 3).

Counterfactual Reasoning. Counterfactuals compare what happened with what *would have* happened. For the commuter: *given that she actually left at time t' and would have taken mode m under the prevailing weather, what would her performance Y have been had she left at time t instead?*—a query $P(Y_t \mid T{=}t', M{=}m)$. The factual and hypothetical are different worlds but concern the same commuter, hence share the same hidden causes. In the categorical approach [8] this is represented via a *twin* (or parallel-world) string diagram: the diagram for the factual world (Figure 1a) is placed beside its $\mathrm{do}(T{=}t)$ surgery (Figure 1b) for the counterfactual, but with the latent state R appearing *once* and feeding both worlds. Sharing this single R couples the worlds, as the deterministic mechanisms then fix the same hidden circumstances across both, while the evidence $T{=}t'$, $M{=}m$ constrains the corresponding output wires. A query is *identifiable* if its value is uniquely determined by the available distribution family (any two SCMs agreeing on the available distributions must agree on the query), and *non-identifiable* otherwise: in the bow graph $X \to Y$, $X \leftrightarrow Y$, the

effect $P(Y \mid \mathrm{do}(X{=}x))$ is not identifiable from $P(X, Y)$ alone. Identifiability is decided by the ID-CF algorithm [16]: after surgery and simplification the diagram splits into maximal confounded blocks (the *c-components*, or districts), whose diagrammatic counterparts are the *R-fragments* the implementation extracts and rewrites next.

Why a Diagrammatic Perspective. The payoff of the categorical account is that everything above lives in *one* language: the model, the counterfactual diagram, and the observable estimand it reduces to are all string diagrams in a Markov category, while an intervention $\mathrm{do}(X{=}x)$ is a local surgery operation transforming one diagram into another—rather than reasoning being split between a combinatorial graph and a separate symbolic do-calculus. Because a diagram is a morphism, not merely a graph, the representation is *compositional*: models are assembled from smaller pieces, and the operations on them—surgery, simplification, identification—are equational rewrites, sound by the laws of the category, so one can reason by structural induction and prove properties of a model—a route to mechanised and, ultimately, formally verified reasoning.

3 Implementation

We implemented our approach [12] in Julia on the Catlab ecosystem [3], which realises symmetric monoidal categories through *wiring diagrams*, the concrete data structure for a string diagram. A model is supplied in its standard graphical form of an *Acyclic Directed Mixed Graph* (ADMG), with directed edges for causal links and bidirected edges $X \leftrightarrow Y$ for hidden common causes (the confounding of Section 2), or directly as a Catlab `WiringDiagram`, in which case the compilation stage below is skipped. All diagrammatic stages manipulate `WiringDiagram`s, in which boxes are mechanisms (morphisms) and wires variable dependencies; each box carries a symbolic name encoding its role—`f_X` a mechanism, `PU_X` an exogenous source, `do_X=x` an intervention, `obs_X=x` an observation, `c_X` a deterministic constant mechanism—so surgery and identification are uniformly rewrites on named boxes and their wiring.

Pipeline, Illustrated on a Worked Example. We follow our *commute* scenario through the pipeline, on the counterfactual query: for someone who left at time t' under weather w, what would their work performance have been had they left at time t, given that under the same weather they take mode m? A single entry point, `identify_counterfactual`, runs five stages, timing each and exporting each intermediate diagram to Chyp:

1. **ADMG compilation.** The input ADMG is turned into the string diagram of Section 2: it is first *rootified*, replacing each bidirected edge $X \leftrightarrow Y$ by a fresh latent root feeding both endpoints, and then each variable yields one mechanism box $f_V : (\mathrm{pa}(V), U_V) \to V$ and one exogenous source PU_V (the noise state ω_V of Section 2). For the commute model, $T \leftrightarrow Y$ becomes the shared latent root R feeding both T and Y.
2. **Parallel-world construction.** The base diagram is lifted into a multiverse by duplicating each mechanism once per world while *sharing* the exogenous

sources (the twin construction of Section 2). An intervened variable splits into a discarded sink and a fresh `do_X=x` source; an observed variable gets an `obs_X=x` box; the factual Y and counterfactual Y_t stay coupled through the shared root R.

3. **Simplification.** Four rewrite passes reduce the diagram to canonical form: pruning discarded branches and dead boxes, separating observations from copy/fan-out, merging deterministic mechanisms with identical input sources, and absorbing exogenous sources into deterministic constant mechanisms where the absorption condition holds.
4. **ID-CF identification.** The simplified diagram is decomposed into *R-fragments*, the diagrammatic counterpart of c-components (districts) in ADMGs, and each fragment is rewritten by case analysis into an interventional probability box $P(C_l, F_l^{out} \mid \mathrm{do}(D_l), \mathrm{do}(F_l^{in}))$. A residual unabsorbed exogenous source signals a non-identifiable query.
5. **Expression output.** The resulting probability boxes are parsed into a small expression tree and returned as a symbolic estimand (rendered in LaTeX), with optional algebraic simplification (e.g. dropping vacuous interventions and eliminating normalised summations).

On finite diagrams each simplification pass terminates, since every rewrite strictly decreases a well-founded structural measure (the number of removable boxes, unresolved copy-map branches, or mergeable mechanism occurrences). Following `simplify-cf` [13], mechanisms are processed in a fixed topological order. We have not proved that arbitrary rewrite orders produce the same verdict, so the implementation always uses this fixed order.

The simplified diagram and identified estimand for this example are in the `README.md` [12] and under the configured trace directory, with full-resolution renders of every stage collected in a companion gallery (`figure-gallery.pdf`) in the repository root. That README, the recommended starting point, maps the source module behind each stage, documents the concrete data types (the input `ADMGModel`, the `ConfoundedModel` and `WiringDiagram` it compiles to, the per-world `CounterfactualQuery`, and the output term language) and the supporting modules (Bayesian-network import, Chyp serialisation [10], graph utilities), and gives a runnable quickstart for this pipeline and the comb disintegration examples below.

Comb Disintegration Causal Inference Extension. The pipeline above already *identifies* interventional effects as the single-world special case of ID-CF; the prototype additionally includes a scoped extension that *computes* them numerically, via comb disintegration [8], exposed through `infer_causal_effect`. Given a string diagram, a finite observational joint table, and a comb structure (context, intervention, bridge, and outcome variables; the full input contract and the smoking and HEPAR2 examples are in the README API), it checks that the diagram admits the comb decomposition and computes $P(C \mid A, \mathrm{do}(X{=}x))$ from the table. The diagram thus plays a purely *structural* role, establishing that comb disintegration validly applies to the model; the interventional probabilities themselves are computed from the separately supplied joint table (attaching a

stochastic map to each box, so that the joint is composed directly, is left to future work). In the pyAgrum comparison below, both tools compute the *same* interventional distribution, ours via the comb decomposition checked on the diagram and pyAgrum via its variable-elimination `causalImpact`; the measured runtimes cover the full numerical computation, not just this structural check.

Design Decisions. From simplification onwards, every state of the pipeline is a single `WiringDiagram`, so identification is pure pattern-matching on box names and rewiring, with no separate symbolic representation; and because the stages mirror the theoretical algorithm one-to-one, each is independently testable via the `test/` suite's per-stage test sets. Keeping each stage explicit and Chyp-exported makes every transformation inspectable—unlike symbolic tools, where decomposition and simplification are internal.

4 Empirical Evaluation

We evaluate along two dimensions: correctness, via a differential-testing harness against `cfid` [17] (a published R implementation of ID-CF on ADMGs), and runtime, against `cfid` and Python `y0` [6].

Differential Correctness Testing. The harness generates counterfactual identification problems, translates each into the Julia and R `cfid` input formats, and compares the ID/FAIL verdicts (ID: an identifying estimand found; FAIL: no identifying reduction found—not a proof of non-identifiability). The corpus mixes structured benchmark cases with randomly sampled bow-free ADMGs over 5–8 nodes, each paired with a conditional counterfactual query; cases reducing to single-world interventional queries are filtered out. We use `cfid` 0.1.9 in the interventional-data setting, matching the implementation's scope, and select it because its counterfactual-query and interventional-data interfaces align directly with ours; `y0` is not included because it uses a different binary-event input format, for which we have not established a semantics-preserving translation of all generated queries. `cfid` is treated as a comparison implementation rather than a ground-truth oracle: disagreements are examined separately using ID*/IDC*.

Aggregated over six random seeds (310 cases each, 1860 total; no tool errors, none dropped), Julia agrees with `cfid` on 1832—a 98.5% rate (per-seed 97.7–99.4%). The 28 disagreements split 21 where Julia returns FAIL and `cfid` ID, versus 7 reversed; we inspected all 28 manually using ID*/IDC*. Of the 21 FAIL cases, 20 were confirmed non-identifiable; the remaining one is a false negative of our tool, caused by a missing IDC* evidence-to-action reduction (a conditional observation satisfying the required d-separation criterion can first be exchanged for an intervention, removing the apparent cross-world conflict before identification). The 7 reversed queries were confirmed identifiable, as our tool reports. We therefore found no false-positive identification by our tool in this corpus, but one conservative false negative.

Runtime Benchmarking. Given this agreement, we compare runtime against `cfid` and `y0` (warm medians: 5 warm-up runs then 30 measured repetitions,

excluding Julia's first-call JIT compilation).[1] For identifiable counterfactual queries, all three tools return symbolic estimands, not only Boolean verdicts. We use three standard queries (`drug`, `party`, and a conditional counterfactual query on the real-world `hepar2` network) and synthetic families of controllable size and topology (`chain`, `fanin`, `dense`, and a confounded chain `chain_bi` with bidirected edges).[2] The table below gives warm-median runtimes for the standard queries (top) and the synthetic families at $n = 32$ (bottom); each speedup is the baseline runtime over Julia's (> 1: Julia faster). Since the tools are written in different languages, absolute ratios partly reflect language and implementation differences; the stage-level breakdown below therefore localises cost *within* our pipeline, attributing the scaling to the categorical method rather than to Julia.

Case	Julia (ms)	R `cfid` (ms)	Python `y0` (ms)	R/Julia	`y0`/Julia
`drug`	19.85	53.37	13.33	2.69×	0.67×
`party`	10.15	28.31	4.84	2.79×	0.48×
`hepar2_conditional`	90.84	7812.44	2005.27	86.00×	22.08×
`chain` (n=32)	113.38	694.98	540.64	6.13×	4.77×
`fanin` (n=32)	188.52	714.90	199.77	3.79×	1.06×
`dense` (n=32)	302.95	708.91	474.46	2.34×	1.57×
`chain_bi` (n=32)	454.37	170.59	17.89	0.38×	0.04×

On the standard benchmarks (top), the tool is consistently faster than R `cfid`: ∼2.7× on the small `drug` and `party` queries, 86× on the larger `hepar2_conditional`. `y0` is faster on the small queries (a direct graph-level implementation has lower overhead) but ∼22× slower on `hepar2_conditional`—the explicit simplification pipeline pays off as the model grows.

A stage-level breakdown explains this: the dominant cost is the explicit *simplification* stage (in `drug`, most of the runtime) while identification (Steps 4–5) is lightweight, whereas `cfid` exposes the main computation through one `identifiable` call. The categorical method thus trades a heavier simplification phase for a light identification one: an advantage when simplification substantially reduces the later problem, especially on larger or structurally favourable graphs.

The synthetic families (bottom) show this structure-dependence. Julia outperforms `cfid` on `chain`, `fanin`, and `dense`, and `y0` on `chain` and `dense` (comparable on `fanin`), making it competitive on sparse and moderately connected structures; it reverses only on the heavily confounded `chain_bi`, where bidirected confounding inflates simplification and this preprocessing, not identification, dominates the runtime. These results are not specific to $n = 32$: sweeping $n \in \{8, 16, 24, 32\}$, Julia's advantage over both baselines *grows* with n on `chain`, `fanin`, and `dense`

[1] Environment: Julia 1.11.6 (Catlab 0.17.2); R 4.5.2 (`cfid` 0.1.9); Python 3.14.2 (`y0` 0.2.11, pyAgrum 2.3.2); Intel i7-11800H, 16 GB RAM, Windows 10.

[2] Node/directed-edge/bidirected-edge counts are: `drug` 5/4/1, `party` 4/4/0, and `hepar2` 70/123/0. At $n = 32$, the synthetic counts are: `chain` 32/31/0, `fanin` 32/61/0, `dense` 32/172/0, and `chain_bi` 32/31/31.

(on `chain`, $3.4\times \to 6.1\times$ versus `cfid` and $1.2\times \to 4.8\times$ versus `y0`), while on `chain_bi` it stays below 1 and *worsens* ($0.10\times \to 0.04\times$ versus `y0`).

Comb Disintegration Causal Inference Benchmarks. We also evaluate the comb disintegration extension against pyAgrum's `causalImpact`. This comparison is separate from the previous ones, as `cfid` and `y0` return symbolic formulas whereas this extension returns numerical interventional distributions. We use HEPAR2-derived examples whose selected variables form clinically meaningful exposure–outcome patterns, both tools running on the same variables. Grouped by comb size (speedup = pyAgrum over Julia), Julia is slower only at the smallest comb and increasingly faster as the comb grows:

Group	Comb size	Julia (ms)	pyAgrum (ms)	pyAgrum/Julia
1	3	3.29	2.60	0.79×
2	4	3.82	4.02	1.05×
3	5	4.56	6.55	1.44×
4	6	7.15	13.67	1.91×
5	7	8.11	16.75	2.06×

5 Discussion

We present a prototype for counterfactual identification built on the categorical semantics of string diagram surgery, realising the full ID-CF pipeline as rewrites on Catlab wiring diagrams with a Chyp trace at every stage. The evaluation shows this is not merely feasible but competitive: high verdict-level agreement with `cfid`, and substantially faster runtimes on sparse and moderately dense models, at the cost of a simplification phase that bottlenecks on densely confounded graphs. Four main limitations remain. (i) *Scope*: the tool targets ID-CF under the usual assumptions (a correctly specified finite discrete ADMG with confounding as bidirected edges) and does not yet cover causal discovery or continuous latent variables; the bidirected-edge abstraction records confounding but not the full hidden-variable mechanism, and broadening the query class and integrating probabilistic inference engines are natural next steps. (ii) *Structure-dependent performance*: simplification relies on repeated full-graph traversals and dominates runtime, so on dense and especially bidirected-confounded graphs (e.g. `chain_bi`) this preprocessing can erase or reverse the speedup; incremental rewriting and better wiring diagram data structures would remove these scans. (iii) *Visualisation*: Chyp export aids inspection but has a limited character set (symbols are mangled in labels, most acutely in identification outputs) and does not scale to large diagrams; a dedicated interactive visualiser (splicing, intervention, rewriting) would address this. (iv) *Verification and interoperability*: the explicit diagrams make every rewrite inspectable but are not yet formally verified and lack native integrations; future work includes formal verification, richer categorical and probabilistic toolchains (e.g. EfProb [2]), and links to causal-representation and reinforcement learning.

Acknowledgements. We acknowledge support from the Advanced Research and Invention Agency (ARIA), under its *Safeguarded AI* programme.

References

1. Caldwell, B., Spencer, W., Kissinger, A., Rand, R.: Tensorrocq: Enabling diagrammatic reasoning in rocq (2026), https://arxiv.org/abs/2604.17592
2. Cho, K., Jacobs, B.: The efprob library for probabilistic calculations. In: 7th Conference on Algebra and Coalgebra in Computer Science (CALCO 2017). Leibniz International Proceedings in Informatics (LIPIcs), vol. 72, pp. 25:1–25:8. Schloss Dagstuhl – Leibniz-Zentrum f'ur Informatik, Dagstuhl, Germany (2017). https://doi.org/10.4230/LIPIcs.CALCO.2017.25, https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CALCO.2017.25
3. contributors, T.A.: Catlab.jl: A compositional algebraic modeling library in julia. https://github.com/AlgebraicJulia/Catlab.jl (2026)
4. Friend, I., Kissinger, A., Spekkens, R.W., Wolfe, E.: Compositional causal identification from imperfect or disturbing observations. Entropy **27**(7), 732 (2025). https://doi.org/10.3390/e27070732
5. Fritz, T.: Synthetic approach to markov kernels, conditional independence and theorems on sufficient statistics (2020), preprint
6. Hoyt, C.T., Bakker, C., Callahan, R.J., Cottam, J., George, A., Gyori, B.M., Hummel, H.M., Merrill, N., Mohammad Taheri, S., Navada, P.P., Parent, M.A., Rupe, A., Vitek, O., Zucker, J.: Causal identification with Y_0 (2025). https://doi.org/10.48550/arXiv.2508.03167, https://arxiv.org/abs/2508.03167
7. Jacobs, B., Kissinger, A., Zanasi, F.: Causal inference by string diagram surgery. In: Foundations of Software Science and Computation Structures. Lecture Notes in Computer Science, vol. 11425, pp. 313–329. Springer (2019). https://doi.org/10.1007/978-3-030-17127-8_18, https://doi.org/10.1007/978-3-030-17127-8_18
8. Jacobs, B., Kissinger, A., Zanasi, F.: Causal inference via string diagram surgery: A diagrammatic approach to interventions and counterfactuals. Mathematical Structures in Computer Science **31**(5), 553–574 (2021). https://doi.org/10.1017/S096012952100027X, https://doi.org/10.1017/S096012952100027X
9. Jacobs, B., Zanasi, F.: The logical essentials of bayesian reasoning. In: Foundations of Probabilistic Programming, pp. 295–332. Cambridge University Press (2020). https://doi.org/10.1017/9781108770750.010, https://doi.org/10.1017/9781108770750.010
10. Kissinger, A.: Chyp: A compositional hypergraph library. https://pypi.org/project/chyp/0.2/ (2024), https://pypi.org/project/chyp/0.2/, python package version 0.2
11. Kissinger, A., van de Wetering, J.: Picturing Quantum Software: An Introduction to the ZX-Calculus and Quantum Compilation. Preprint (2024), https://zxcalc.github.io/book/, available online at https://zxcalc.github.io/book/
12. Li, S., Zanasi, F.: The IDCF string-diagram toolkit. https://github.com/GrayratA/idcf-string-diagram-toolkit/tree/artifact-release (2026)
13. Lorenz, D., Tull, J.: Causal reasoning with string diagrams. In: Proceedings of the 20th International Conference on Algebra and Coalgebra in Computer Science (2023)
14. Pearl, J.: Causality: Models, Reasoning, and Inference. Cambridge University Press, 2nd edn. (2009)
15. Piedeleu, R., Zanasi, F.: An introduction to string diagrams for computer scientists (2023), https://arxiv.org/abs/2305.08768, arXiv preprint arXiv:2305.08768
16. Shpitser, I., Pearl, J.: Complete identification methods for the causal hierarchy. Journal of Machine Learning Research **9**, 1941–1979 (2008)

17. Tikka, S.: The r journal: Identifying counterfactual queries with the r package cfid. The R Journal **15**, 330–343 (2023). https://doi.org/10.32614/RJ-2023-053, https://doi.org/10.32614/RJ-2023-053
18. Tull, S., Kleiner, J., Smithe, T.S.C.: Active inference in string diagrams: A categorical account of predictive processing and free energy (2023), https://arxiv.org/abs/2308.00861, arXiv preprint arXiv:2308.00861